

Finding Regularity:

High-Performance Computing for Graph Algorithms

Gökhan Gökürk

High-performance computing and parallel algorithms

Computer Science Seminar

Research background

Sabancı University

BSc · MSc · PhD · PostDoc

Research

- High-performance computing
- Parallel and distributed algorithms
- GPU and multi-GPU computing
- Graph and probabilistic algorithms
- Performance engineering

Technical consulting

- Şişecam
- Akbank
- Sigorta Bilgi Merkezi
- Aksigorta

High-Performance Computing (HPC)

Designing **algorithms and systems** to solve demanding computational problems efficiently.

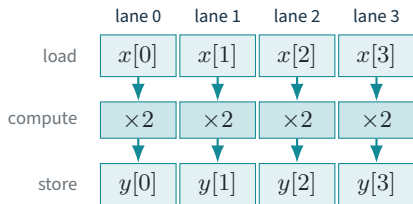
- **Algorithmic choices:** reduce work, change representations, reorganize execution.
- **Hardware costs:** moving data and keeping parallel units productive.
- **Application requirements:** preserve the result or required accuracy.

More arithmetic can be worthwhile if it avoids expensive data movement.

Why parallelism is not automatically performance

A **lane** is one position in a group performing an operation in parallel.

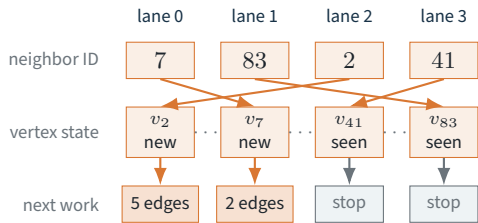
Regular array update: $y[i] = 2x[i]$



Predictable, direct access

Adjacent array elements; equal work per lane.

Graph traversal: data-dependent addresses



Unpredictable, indirect access

Scattered vertex state; unequal work per lane.

More lanes help when data access and work distribution keep them productive.

The research question

How can we reorganize irregular computation so that modern parallel hardware can execute it efficiently?

Regularity: predictable memory access and similar work across parallel lanes.

What can I change?

What can it improve?

Which work runs together

Reuse data; bring memory accesses closer together.

What state is stored

Reduce the memory footprint; make updates uniform.

Where work executes

Match work to CPU, GPU, and device-local memory.

Each change must preserve the required result, sampling distribution, or accuracy target.

Running example: influence maximization

Which k seeds maximize their combined influence?

Spread is stochastic; influence overlaps.

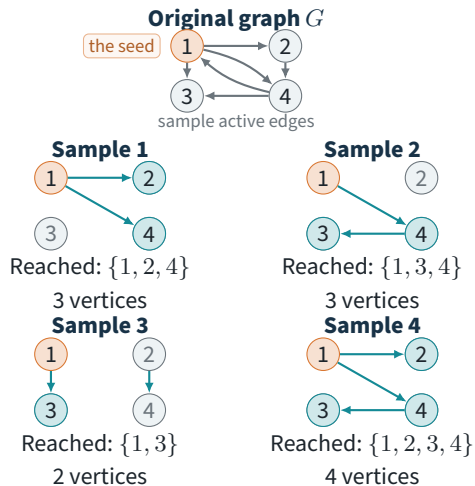
Estimate the influence of seed 1

1. Sample the active edges.
2. Follow them from the seed.
3. Count vertices, including the seed.

Average the four counts

$$\frac{3 + 3 + 2 + 4}{4} = 3 \text{ vertices}$$

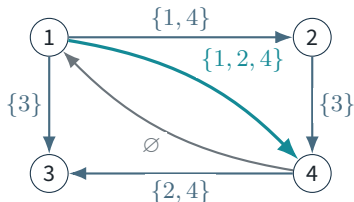
Repeat for candidate seeds and simulations.



Hash-based sampling on a shared graph

$$(u, v) \in G_r \iff \mathbf{X}_r \oplus h(u, v) \leq T_{uv}$$

Sample memberships on one graph



Highlighted edge $1 \rightarrow 4$: $h(1, 4) = 6$

	$r = 1$	$r = 2$	$r = 3$	$r = 4$
Sample seeds X_r	5	4	14	7
$X_r \oplus 6$	3	2	8	1
Active? (≤ 7)	yes	yes	no	yes

Reconstructed membership: $\{1, 2, 4\}$.

**One sampling rule enables reconstruction and vectorized updates.
Its hash structure also enables better scheduling, as we will see later.**

Memberships illustrate the samples on slide 6; they need not be stored. Example uses 4-bit values.

Directed edges use ordered endpoints; undirected hashes are symmetric. Repeatability alone does not prove independent edge sampling.

Sample-major versus vertex-major storage

Same batch: vertex 4 in four samples, one sample per lane.

Each row is contiguous in memory; rows are stored consecutively.

Sample-major: state[r][v]

	v = 1	v = 2	v = 3	v = 4	
r = 1				lane 0	...
r = 2				lane 1	...
r = 3				lane 2	...
r = 4				lane 3	...

The batch spans four sample rows.

Stride between lanes: $|V|$ state entries.

Vertex-major: state[v][r]

	r = 1	r = 2	r = 3	r = 4	
v = 1					...
v = 2					...
v = 3					...
v = 4	lane 0	lane 1	lane 2	lane 3	...

The batch occupies one contiguous row.

Stride between lanes: 1 state entry.

Vertex-major storage makes the sample-lane batch contiguous.

Both layouts hold the same $|V| \times R$ entries. The sampling mask determines which lanes update.

Contiguous state supports SIMD loads and adjacent GPU lane accesses; physical transaction counts depend on hardware.

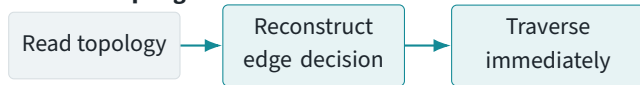
Compute more, move less

Sampling can happen inside the traversal.

Materialized samples



Fused sampling



Extra XORs and comparisons replace stored edge decisions.

Cheap arithmetic can remove a more expensive memory operation.

Three graphs where the reference completed, $K = 50$. Samples run separately in this comparison; sample batching is an additional change.

~**3-21**×

Reported speedup
from fused sampling

Sequential comparison:
FusedSampling vs. MixGreedy

The next bottleneck: per-sample state

Sharing the graph still leaves a state entry for every vertex and sample.

The sample-state matrix remains

	$r = 1$	$r = 2$	$r = 3$	$r = 4$	
$v = 1$	$s_{1,1}$	$s_{1,2}$	$s_{1,3}$	$s_{1,4}$	$\dots$
$v = 2$	$s_{2,1}$	$s_{2,2}$	$s_{2,3}$	$s_{2,4}$	$\dots$
$v = 3$	$s_{3,1}$	$s_{3,2}$	$s_{3,3}$	$s_{3,4}$	$\dots$
$v = 4$	$s_{4,1}$	$s_{4,2}$	$s_{4,3}$	$s_{4,4}$	$\dots$

$|V| \times R$ state entries

What does the influence query use?

Sample 1 from our running example:

$$\text{Reach}_{G_1}(\{1\}) = \{1, 2, 4\}$$

$$|\text{Reach}_{G_1}(\{1\})| = 3$$

Could we estimate the count with smaller state?

The next change is the representation of each entry in that matrix.

Compact summaries still need sample-aligned state. They introduce estimation error rather than eliminate the sample dimension.

Component labels and count-distinct sketches

MixGreedy: component labels

Undirected graph: initialize $\ell_r(v) = v$.



Converge to label 1.

Merge labels until convergence:

$$\ell_r(v) \leftarrow \min_{x \in \{v\} \cup N_{Gr}(v)} \ell_r(x)$$

Component identity supports exact reach counts.

HyperFuseR: count-distinct sketches

Hash reached vertices; retain rare hash patterns.

Vertex	Hash prefix	Leading zeros
--------	-------------	---------------

2	1...	0
---	------	---

4	001...	2
---	--------	---

7	0001...	3
---	---------	---

Register: $\max(0, 2, 3) = 3$.

Seeing vertex 4 again leaves it at 3.

Register values support a cardinality estimate.

My methods accelerate propagation across samples; sketches reduce the state per entry.

Component computation comes from MixGreedy. A sketch register is not an exact count; sketching introduces estimation error.

Sketch propagation across sample lanes

Merge corresponding registers where the sampled edge is active.

	$r = 1$	$r = 2$	$r = 3$	$r = 4$
M_u	2	1	3	2
M_v	4	3	1	5
active?	1	0	1	1
new M_u	4	1	3	5

Active edge: $M_u[r] \leftarrow \max(M_u[r], M_v[r])$.

Inactive edge: keep $M_u[r]$.

Compact registers turn propagation into masked vector maximums.

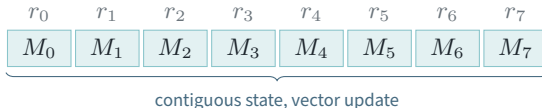
Repeated information leaves a register unchanged. The method monitors estimation error and rebuilds summaries when needed.

Mapping the sample dimension to hardware

The work unit is still one edge across several samples.

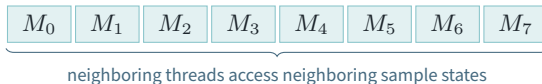
CPU

SIMD lanes



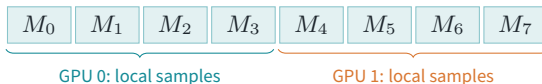
GPU

threads in a warp



Multiple GPUs

sample partitions



Across devices: propagate locally, then combine sketch statistics for source selection.

Eight sample entries shown schematically. Vector widths, warp widths, and device partition sizes differ.

FASST: sample order changes hardware utilization

For this edge, samples 2, 4, 6, and 8 are active.

Original assignment

warp A	r_1	r_2	r_3	r_4
warp B	r_5	r_6	r_7	r_8

Both warps enter the update.

After sorting sample keys

warp A	r_4	r_2	r_6	r_8
warp B	r_7	r_5	r_1	r_3

One warp updates. One skips the edge.

DiFuser's FASST sorts the X_r in $\mathbf{X}_r \oplus \mathbf{h}(u, v) \leq T_{uv}$.

Each sample's state follows its key; its edge decisions stay the same.

Grouping inactive samples can remove an entire warp's update work.

Colored cells are active. Four-lane toy warps illustrate one edge; sorting keys does not guarantee this grouping for every edge.

Sample-space decomposition scales across GPUs

Reported strong scaling

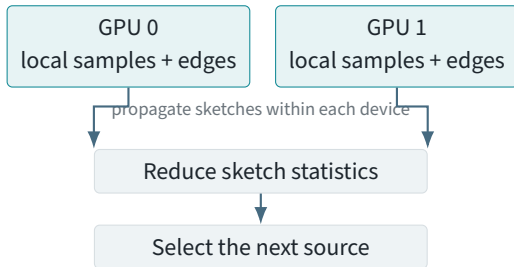
A100 GPUs	Speedup
1	1.00×
2	2.45×
4	4.28×
8	5.64×

Same total sample workload.

Partition samples so traversal stays local; communicate at global decisions.

Geometric means, $K = 50$, $J = 1024$; two GPUs per node for multi-GPU runs. Friendster excluded because the one-GPU configuration does not fit.

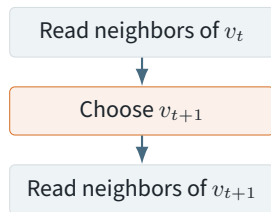
What stays local? What communicates?



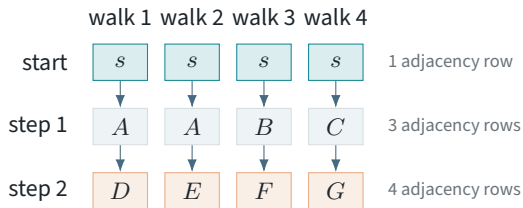
A harder case: random walks destroy exact alignment

Every next adjacency address depends on the previous choice.

Within one walk



Across four independent walks



Can we group walks that are likely to request the same adjacency data?

Illustrative paths. Walks have independent state, but each individual path is a chain of dependent memory accesses.

SABA: seed ordering creates opportunities for reuse

Same source, same eight random keys, different groups of four.

First choice: $[0, .25) \rightarrow A$ $[\cdot25, .50) \rightarrow B$ $[\cdot50, .75) \rightarrow C$ $[\cdot75, 1) \rightarrow D$

Unordered walks

key	.83	.14	.63	.29
next vertex	<i>D</i>	<i>A</i>	<i>C</i>	<i>B</i>

key	.91	.48	.05	.72
next vertex	<i>D</i>	<i>B</i>	<i>A</i>	<i>C</i>

4 distinct adjacency rows per group

Sorted walks: two bouquets

key	.05	.14	.29	.48
next vertex	<i>A</i>	<i>A</i>	<i>B</i>	<i>B</i>

key	.63	.72	.83	.91
next vertex	<i>C</i>	<i>C</i>	<i>D</i>	<i>D</i>

2 distinct adjacency rows per group

Nearby keys can share early adjacency reads; each walk keeps its own residual state.

A bouquet is a batch of walks processed together. Toy keys are normalized independent draws. Later steps can diverge.

Correct sampling and useful grouping

The next neighbor must be uniform, even after observing the path so far.

Example: 12 equally likely inputs, 3 neighbors, 4 inputs per neighbor.

input x	0	1	2	3	4	5	6	7	8	9	10	11
neighbor k	0				1				2			
residual x'	0	1	2	3	0	1	2	3	0	1	2	3

For $x = 7$: choose neighbor 1; retain $x' = 3$ in a range of size 4.

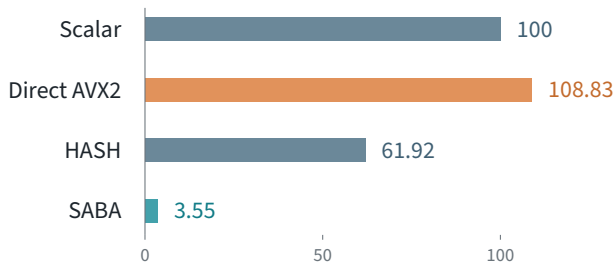
Every chosen neighbor leaves the same uniform residual range $\{0, 1, 2, 3\}$.

Use the interval to choose a neighbor; keep only the unused position inside it.

If the range is exhausted or the input lies outside the equal-sized buckets, switch permanently to fresh unbiased sampling. Assumes independent uniform inputs; full rule in backup.

Parallelism without locality can increase cache misses

Last-level cache misses, normalized to scalar walks = 100.



Direct vectorization

8.83% more cache misses

SABA vs. HASH

3.38×

Faster walk processing

Scheduling changes which data the parallel lanes request together.

Reported earlier sampler implementation, not measurements of the corrected residual sampler. Geometric means over 10 graphs; 16,384 walks/vertex, Broadwell Xeons. Runtime comparison: 16 threads.

A methodology for irregular computation

Each project changes the work responsible for a specific machine cost.

Observed cost	Algorithmic change	Architectural effect
Repeated graph reads	Hash-based edge sampling; batched updates	Reuse topology and avoid stored samples
Large exact state	Estimate reach with mergeable registers	Smaller entries and uniform updates
Inactive sample lanes	Exploit XOR sampling: sort keys, partition samples	Whole-batch skips and device-local work
Unrelated walk addresses	Arrange trajectories using sampling structure	More opportunities to reuse adjacency

Change execution order, representation, or the sampling construction.

When restructuring pays off

A transformation has to save more time than it adds.

Choice	Potential saving	Added cost or limit
Larger sample batches	Reuse topology across more work	More state and inactive lanes
Order trajectories	Reuse early adjacency data	Sorting cost; later paths separate
Compact sketches	Less state traffic	Estimation error and rebuilding
Place work on a GPU	More concurrent execution	Transfers and enough work to occupy it

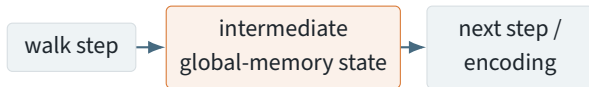
Measure total execution time at the required accuracy.

These competing costs motivate an adaptive choice of batch size, ordering, representation, and processor.

Ongoing work: fused walks for graph learning

NeuralBloom: constructing NeuralWalker's walks and structural encodings.

Separate GPU operations



Fused execution

One thread advances a walk and builds its encodings
Private state stays within the kernel

Write the required outputs for the neural model.

The systems question: how much work can stay inside one kernel?

Ongoing work. Timing covers walk computation, not complete model training. Fusion depth must account for register pressure and available parallelism.

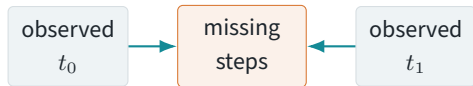
~2×

Preliminary speedup
in walk computation

Same model-facing
representation.

Ongoing work: reconstruction and trajectory execution

Battus: infer missing diffusion states



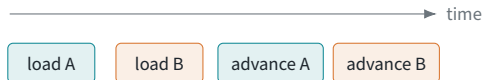
Both observations constrain the reconstruction.

Per-vertex probabilities and forward-backward inference become repeated graph passes.

Which inference stages can we remove while retaining useful accuracy?

Both projects are ongoing. Battus changes the approximation; deterministic trajectories need their own accuracy analysis. The timeline is schematic.

Trajectories: overlap independent work



Independent state allows interleaved execution.

Deterministic path coordinates, CPU interleaving, and a GPU implementation.

Can we hide memory latency when walks have little locality?

Finding regularity systematically

Can a runtime choose how to batch and schedule irregular work?

First project: adaptive scheduling

Observe a short batch	Choose an execution strategy
Repeated vertices	Group walks for adjacency reuse
Few shared addresses, long stalls	Interleave independent trajectories
Little predicted benefit	Keep the current order

Start with graph walks and NeuralBloom; extend to representation and CPU-GPU placement.

Longer-term workloads: sparse and dynamic graphs, sampling-heavy ML systems, and scientific Monte Carlo. Each extension needs its own cost model and correctness conditions.

What would count as success?

Beat tuned static schedules on unseen workloads, including observation and scheduling costs.

What must the application specify?

Dependencies, random-state ownership, and the required accuracy.

Finding Regularity

- | | |
|------------------------------|---|
| Across samples | Read shared topology once for several updates. |
| In the representation | Replace exact state with compact, uniform merges. |
| In the schedule | Bring walks together when they can reuse data. |

My research connects algorithm design to memory behavior and parallel execution.

Next: predict and choose these transformations automatically.

Questions

Backup slides

Backup: fused-sampling implementation in InFuseR-MG

For undirected graphs, use a symmetric edge hash:

$$h(u, v) = \text{MURMUR3}(\min(u, v) \parallel \max(u, v))$$

so $h(u, v) = h(v, u)$.

For simulation r :

$$\rho(u, v)_r = \frac{X_r \oplus h(u, v)}{h_{\max}}$$

and edge (u, v) is active iff

$$\rho(u, v)_r \leq w_{u,v}.$$

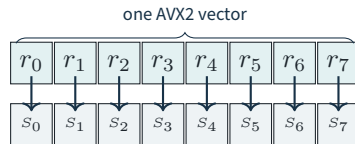
The system can reconstruct membership from edge/sample identity without materializing sampled graphs.

Reconstructibility does not by itself prove joint independence across edges. This edge-membership construction is not a proof of random-walk correctness.

Backup: SIMD implementation details

AVX2 processes eight 32-bit sample states at a time.

1. compare component labels
2. broadcast precomputed edge hash
3. XOR with simulation seeds
4. compare against probability threshold
5. masked / blended updates
6. movemask-style reduction for changed lanes



Per-vertex sample state is contiguous, so one vector load fetches several simulations for the same vertex.

Eight 32-bit lanes per AVX2 vector. SIMD accelerates the sample-state update after the layout change.

Backup: HyperFuseR sketch details

Flajolet–Martin style register update:

$$M[j] = \max(M[j], \text{clz}(h_j(x)))$$

Sketch merge:

$$M_{uv}[j] = \max(M_u[j], M_v[j])$$

M_u	2	5	1	4	3	6	2	5
M_v	3	4	6	4	7	2	2	8
max	3	5	6	4	7	6	2	8

Compact registers estimate cardinality; they do not preserve exact reached identities.

Illustrative registers. All shown merges are active; inactive sample lanes retain their previous value.

Backup: error-adaptive rebuilding in HyperFuseR

After selecting more sources, retained sketches can become less useful for estimating marginal reach.

1. Estimate a candidate's marginal gain from compact state.
2. Compare that estimate with a Monte Carlo evaluation.
3. Rebuild when local and global discrepancy criteria trigger.

The paper uses local threshold $\epsilon_l = 0.3$ and global threshold $\epsilon_g = 0.01$ in its experiments.

These empirical rebuilding thresholds are not universal error guarantees.

Rebuilding responds to discrepancies between sketch estimates and Monte Carlo evaluations.

Backup: FASST sample keys and device-local edges

The ordering uses the fused edge-decision construction.

$$\text{active}(e, r) = [X_r \oplus h(e) < T_e]$$

Sort the existing keys X_r once; carry the associated sample state with them.

Why grouping can help

Nearby keys have related high-order bits. For a fixed edge hash and threshold, their decisions can cluster.

A warp with no active sample skips the update.

Device-local edge set

For device p with sample identities $\mathcal{R}_p$:

$$E_p = \{e : \exists r \in \mathcal{R}_p, \text{active}(e, r)\}$$

An edge unused by all local samples can be omitted. Device edge sets may overlap.

Sorting changes sample placement; it retains the sampled keys and decisions.

Clustering depends on the decision construction and edge probabilities. It is not guaranteed by an arbitrary identity-based hash.

Backup: multi-GPU scaling

GPUs	Observed speedup	Ideal
1	1.00×	1×
2	2.45×	2×
4	4.28×	4×
8	5.64×	8×

Scaling limits: edge overlap, load imbalance, and global reductions. Partitioning can also reduce local work and memory footprint.

Geometric means; $K = 50$, $J = 1024$; two A100 GPUs per node for multi-GPU runs. Friendster excluded.

Backup: performance comparisons and their scope

Result	Reported value	What it demonstrates
InFuseR-MG vs. state of the art	2.3–173.8×	overall effectiveness
Fused sampling contribution	~3–21×	on-the-fly sampling comparison
HyperFuseR	up to 119×	compact-state approach
DiFuseR, 1 × A100	3.15 × geom. mean vs. gIM	GPU mapping
DiFuseR, 8 GPUs	5.64 × geom. mean vs. 1 GPU	sample-space scale-out
Historical SABA, 16,384 walks/vertex	6.76 × geom. mean vs. naïve	locality-aware vectorized walks
Historical SABA vs. matched hash base-line	3.38×	scheduling contribution

Interpretation rule

SABA rows describe the earlier sampler and do not validate the corrected residual-range method. Other rows compare complete methods unless explicitly identified as fusion or scaling.

Backup: residual-range sampler and unbiased fallback

Initial seed: $X_W \sim \text{Uniform}\{0, \dots, 2^{64} - 2\}$; set $x = X_W, R = 2^{64} - 1$.

$$q = \lfloor R/d \rfloor, \quad \text{if } q > 0 \text{ and } x < dq: \quad k = \lfloor x/q \rfloor, \quad x \leftarrow x \bmod q, \quad R \leftarrow q.$$

Otherwise switch permanently to conventional sampling. For $H = 2^{32}$ and $1 \leq d < H$:

$$t = H \bmod d, \quad z \sim \text{Uniform}\{0, \dots, H - 1\}, \quad p = zd.$$

Reject and redraw while $p \bmod H < t$; otherwise choose $k = \lfloor p/H \rfloor$.

Proof idea: in residual mode each pair (k, r) has one preimage $x = kq + r$. In fallback, each neighbor has equally many accepted inputs. Both modes preserve the conditional transition law.

Assumptions: independent uniform seeds and fresh independent fallback outputs.

Degree-zero vertices terminate. Practical PRNGs approximate the ideal independent-input model.

Backup: sampler validity, ordering, and replay

1. **Sampler validity**

Uniform next neighbor conditional on the labeled walk's path history.

2. **Estimator invariance**

For fixed walks, $\frac{1}{N} \sum_j X(W_j) = \frac{1}{N} \sum_j X(W_{\pi(j)})$ in exact arithmetic.

3. **Exact replay across schedules**

Requires initial seeds and fallback streams owned by walk identity.

The manuscript's implementation initializes fallback PRNG states per lane after sorting; a schedule change can change a labeled walk's realized path.

Distributional validity does not imply identical realized samples or bitwise-identical reductions.

Sorted lane ranks are dependent. Conditional uniformity is a property of labeled walks, before conditioning on sorted rank.

Backup: SABA locality diagnostic

For bouquet width B , effective pool size β , and distinct items η , an independent-uniform occupancy approximation gives:

$$\frac{\mathbb{E}[\eta]}{B} = \frac{\beta}{B} \left(1 - \left(1 - \frac{1}{\beta} \right)^B \right)$$

approximately

$$\frac{\mathbb{E}[\eta]}{B} \approx \frac{\beta}{B} \left(1 - e^{-B/\beta} \right).$$

How to use it

A trend diagnostic for within-bouquet occupancy and potential reuse – not a bound for the actual state-dependent, seed-sorted walks.

Backup: selected work behind the seminar

1. **Boosting Parallel Influence-Maximization Kernels for Undirected Networks with Fusing and Vectorization**
fused sampling, SIMD, multicore execution
2. **Fast and Error-Adaptive Influence Maximization based on Count-Distinct Sketches**
compact state, sketch propagation, adaptive rebuilding
3. **DiFuseR: A Distributed Sketch-based Influence Maximization Algorithm for GPUs**
GPU sample-space execution, FASST, multi-GPU decomposition
4. **Accelerating Spanning-Centrality Approximation with Sampling-Aware Vectorized Random Walks**
SABA, trajectory scheduling, locality-aware vectorization

Local source PDFs and revised SABA manuscript are listed in [papers/README.md](#).

Backup: when bouquet locality persists

Sorting creates an opportunity for reuse; the graph determines how long it lasts.

Factor	Useful locality	Loss of locality
Initial grouping	Shared source and nearby random keys	Keys straddle neighbor intervals
Visit distribution	Many walks revisit the same vertices	Choices spread over many different neighbors
Walk depth	Early residuals retain useful grouping	Different degrees, paths, and fallback decisions separate lanes

A frequently visited hub can help reuse; a large uniform neighbor list can scatter the next choices.

Degree alone does not predict the benefit. Sampler validity is separate from this locality heuristic.

Backup: what each current project must preserve

NeuralBloom

Fused GPU construction of NeuralWalker's walk indices, edge indices, and structural encodings. Preserve the model interface and intended sampling semantics.

Battus

Mean-field forward-backward inference and deterministic decoding. Assess reconstruction quality and graph-causal feasibility separately; this changes the approximation.

Trajectory prototype

Deterministic starting coordinates with CPU interleaving and a Metal GPU implementation. Agreement between implementations is distinct from agreement with an independent random-walk distribution.

Ongoing work; source descriptions in the NeuralBloom, Battus, and randomwalk repositories.

Backup: trajectory coordinates and interleaving

The current deterministic prototype initializes walk i with $x_i = i/N$. At a vertex of degree $d > 0$:

$$k = \lfloor dx \rfloor, \quad x \leftarrow dx - k, \quad v \leftarrow \text{neighbor}(v, k).$$

- CPU: interleave four independent trajectory state chains.
- GPU: one trajectory per thread; exact rational-coordinate arithmetic in Metal.
- Evaluate endpoint approximation separately from execution speed.

Deterministic trajectory rules require a different argument from IID Monte Carlo sampling.

This deterministic prototype does not replenish coordinate precision with fresh randomness.